

Especificaciones Técnicas (SPEC-First) - Kamaleon/Meleon

Esta documentación formaliza los hallazgos del dashboard de Kamaleon en especificaciones técnicas listas para desarrollo, siguiendo estrictamente la sintaxis EARS (*Ubiquitous, Event-driven, State-driven, Unwanted, Optional*).

Módulo 1: Orquestación de UI Server-Driven (Feature Builder)

Referencia: Análisis del componente "Dropdown: Forma de Pago" y el binding SQL en los archivos adjuntos.

SPEC-001: Renderizado Dinámico de UI y Binding SQL

El sistema debe permitir la definición de interfaces móviles desde el servidor mediante manifiestos JSON, eliminando la necesidad de *releases* binarios para cambios de UI.

Requirements (EARS)

- **(Ubiquitous):** El sistema **siempre** debe renderizar los componentes de la interfaz basándose exclusivamente en la definición del "Manifiesto JSON" recibido durante la sincronización.¹
- **(Event-driven): CUANDO** un componente tipo dropdown es instanciado en la vista, **ENTONCES** el motor móvil debe ejecutar la consulta definida en `data_source.query` contra la base de datos SQLite local (`DEVICE_*`).¹
- **(State-driven): SI** la consulta SQL contiene parámetros contextuales (ej. `@current_client_type`), **ENTONCES** el sistema debe inyectar el valor de la sesión del usuario actual antes de la ejecución.¹
- **(Unwanted):** El sistema **no debe** permitir la ejecución de consultas que no estén pre-validadas por el módulo Guardian IA.¹

Constraints

- **Latencia de Renderizado:** La ejecución de la query y el renderizado del control no deben exceder los **100ms** para mantener 60fps.

- **Disponibilidad:** La UI debe ser funcional 100% en modo **Offline**.

Success Criteria

- El dropdown muestra opciones diferentes para el usuario "Retail" vs "Distribuidor" usando el mismo JSON.¹
- Cero excepciones por "Table not found" en producción (garantizado por Guardian IA).

Test Scenarios

- **TC-1.1: Context Injection**
 - *Precondición:* Usuario logueado como "Vendedor".
 - *Input:* Cargar pantalla de Pedido.
 - *Expected:* El dropdown ejecuta WHERE tipo_cliente = 'Vendedor' y muestra solo precios minoristas.
- **TC-1.2: Offline Resilience**
 - *Input:* Activar "Modo Avión" y abrir el formulario.
 - *Expected:* Los datos cargan correctamente desde SQLite local sin error de red.

Módulo 2: Middleware y Validación Externa (Gateway)

Referencia: Triggers de validación de identidad (RENIEC) y logs de monitoreo.

SPEC-002: Validación Síncrona de Identidad (Event Trigger)

El sistema debe interceptar la captura de datos críticos para validarlos contra fuentes de verdad externas antes de permitir su almacenamiento.

Requirements (EARS)

- **(Event-driven): CUANDO** el foco del usuario sale del campo "DNI" (ON_EXIT), **ENTONCES** el Gateway debe invocar el servicio Service.RENIEC con el valor capturado.¹
- **(State-driven): SI** la respuesta del servicio retorna valid: false, **ENTONCES** el sistema debe bloquear el envío del formulario (Block Submit) y mostrar un mensaje de error al usuario.¹
- **(Optional): DONDE SEA POSIBLE**, el sistema debe autocompletar los campos "Nombre" y "Apellido" con la data retornada por el servicio RENIEC para reducir el tiempo de digitación.
- **(State-driven): SI** el servicio externo excede el tiempo de espera configurado (Timeout),

ENTONCES el sistema debe retornar un estado 408 Timeout y permitir el ingreso manual marcando el registro como "No Verificado".¹

Constraints

- **Timeout:** Máximo **2000ms** por llamada externa antes de fallar.¹
- **Latencia Objetivo:** Promedio de **145ms** para validaciones de identidad.¹

Success Criteria

- Tasa de bloqueo de DNIs falsos: 100%.
- Latencia promedio percibida por el usuario < 200ms.

Test Scenarios

- **TC-2.1: Validación Exitosa**
 - *Input:* Ingresar DNI válido y cambiar de campo.
 - *Expected:* El nombre se autocompleta en <200ms y el botón "Guardar" se habilita.
- **TC-2.2: Manejo de Timeout**
 - *Condition:* Simular latencia de red > 2s.
 - *Input:* Ingresar DNI.
 - *Expected:* Mensaje "Sin conexión con RENIEC" y permitir digitación manual.



Módulo 3: Seguridad Predictiva (Guardian IA)

Referencia: Detección de inyecciones SQL y validación de reglas de negocio con RAG.

SPEC-003: Firewall de Código SQL con IA (RAG)

El sistema debe impedir el despliegue de código SQL malicioso o que viole reglas de negocio utilizando análisis estático potenciado por IA.

Requirements (EARS)

- **(Ubiquitous):** El sistema **siempre** debe analizar cualquier script SQL (Triggers, Queries) mediante el motor Guardian IA antes de permitir su distribución a los dispositivos.¹
- **(Event-driven):** **CUANDO** Guardian IA detecta patrones destructivos (ej. DROP, TRUNCATE, ALTER), **ENTONCES** debe rechazar el *release* inmediatamente y registrar el evento "MOMOP TABLE detected".¹
- **(State-driven):** **SI** el script define una regla de negocio (ej. un descuento), **ENTONCES** el motor RAG debe consultar la "Base de Conocimiento Vectorial" (ej.

politica_precios.pdf) para verificar su conformidad.¹

- **(Unwanted):** El sistema **no debe** permitir que un desarrollador con rol "Junior" publique cambios directos en tablas maestras (DEVICE_CUSTOMER) sin aprobación de IA.¹

Constraints

- **Tiempo de Validación:** El análisis debe completarse en menos de **1.0 segundo** (Benchmark actual: 0.3s).¹
- **Falsos Positivos:** < 0.1% para no bloquear el flujo de desarrollo.

Success Criteria

- 100% de rechazo ante intentos de DROP TABLE.
- Feedback en lenguaje natural al desarrollador explicando la razón del rechazo (ej. "El descuento excede el 15% permitido por política").

Test Scenarios

- **TC-3.1: Inyección SQL Destructiva**
 - *Input:* Intentar guardar query DROP TABLE DEVICE_ORDER;
 - *Expected:* Error crítico "MOMOP TABLE detected", bloqueo de guardado.
- **TC-3.2: Violación de Política de Negocio**
 - *Input:* Crear trigger con SET discount = 0.50 (50%).
 - *Context:* PDF de política dice "Máximo 15%".
 - *Expected:* Guardian IA rechaza con "Violación de política: Descuento excede límite".

Módulo 4: Motor de Impresión (Ticket Designer)

Referencia: Generación de tickets JSON y renderizado ESC/POS.

SPEC-004: Generación Dinámica de Documentos Térmicos

El sistema debe generar comprobantes físicos utilizando una estructura lógica JSON que se popula con datos locales en el momento de la impresión.

Requirements (EARS)

- **(Ubiquitous):** El sistema **siempre** debe construir el layout de impresión iterando sobre el array components del template JSON asignado al cliente.¹
- **(Event-driven):** **CUANDO** el motor procesa un componente de tipo items, **ENTONCES** debe ejecutar la consulta SQL definida en source (SELECT * FROM pedido_items...) para

generar las filas del ticket.¹

- **(State-driven):** **SI** la variable {{tenant_ruc}} está presente en el template, **ENTONCES** debe ser reemplazada por el RUC almacenado en la tabla DEVICE_CUSTOMER local.¹
- **(Event-driven):** **CUANDO** el tipo de componente es qr_code, **ENTONCES** el sistema debe generar un bitmap QR válido basado en la concatenación de los datos fiscales del pedido.¹

Constraints

- **Formato:** Compatible con impresoras térmicas ESC/POS de **80mm**.¹
- **Performance:** Generación y envío al buffer de impresión en < 2 segundos.

Success Criteria

- El ticket impreso coincide estructuralmente con el "Preview Térmico" del dashboard.
- Los montos impresos ({{pedido.total}}) coinciden exactamente con la base de datos local.

Test Scenarios

- **TC-4.1: Impresión de N Items**
 - *Input:* Pedido con 50 ítems distintos.
 - *Expected:* El ticket se genera completo, sin cortar texto y con el total correcto al final.
- **TC-4.2: Datos Dinámicos**
 - *Input:* Cambiar el nombre del cliente en DEVICE_CUSTOMER.
 - *Expected:* La siguiente impresión muestra el nuevo nombre sin necesidad de rediseñar el template.

Módulo 5: Sincronización de Datos (Sync Engine)

Referencia: Tablas DEVICE_* e intervalos de sincronización.

SPEC-005: Sincronización Diferencial Priorizada

El sistema debe gestionar la transmisión de datos priorizando la información crítica (Pedidos) sobre la información estática (Catálogos) para optimizar batería y ancho de banda.

Requirements (EARS)

- **(State-driven):** **SI** la tabla es DEVICE_ORDER (Alta prioridad), **ENTONCES** el intervalo de sincronización debe ser de **5 minutos** (Sync: 5m).¹
- **(State-driven):** **SI** la tabla es DEVICE_PRODUCT (Catálogo pesado), **ENTONCES** la

sincronización debe ocurrir cada **2 horas** o bajo demanda (Sync: 2h).¹

- **(Ubiquitous):** El proceso de sincronización **siempre** debe ejecutarse en segundo plano (Background Service) para no bloquear la interacción del usuario.¹
- **(Unwanted):** El sistema **no debe** re-descargar registros que no hayan cambiado (debe usar hash/versión para sync diferencial).

Constraints

- **Velocidad de Transferencia:** Promedio esperado de **45 MB/s** en redes 4G/5G.¹
- **Integridad:** Tasa de éxito de inyección en BD > 99%.¹

Success Criteria

- Un pedido realizado en el móvil aparece en el dashboard central en < 6 minutos.
- La batería del dispositivo no se drena más del 5% por hora debido a procesos de sync.

Test Scenarios

- **TC-5.1: Recuperación tras Offline**
 - *Input:* Realizar 10 pedidos sin internet -> Conectar a internet.
 - *Expected:* Los 10 pedidos se suben automáticamente en el siguiente ciclo de 5m sin intervención.
- **TC-5.2: Actualización de Precios**
 - *Input:* Cambiar precio en servidor -> Forzar sync de catálogo.
 - *Expected:* El nuevo precio se refleja en la tabla DEVICE_PRODUCT_PRICE local.

Fuentes citadas

1. [screenshot-mobilityappsinc-mx-kamaleon-orquestador-demo-html-2026-01-04-12_24_58.pdf](#)